

Backend Configuration

Installation: To run react project you will need couple thing to setup first, Install **node** **recomended version** from [here](#) if you already not installed.

NPM Packages:

- After downloading the template, unzip it
- Open the Terminal.
- Go to the folder (backend): (**cd path/to/template**)
- Run: **npm install**, it will install all used packages for this backend.
- Make sure that there is no error.

After the npm packages are installed, make sure that you have a package.json file and check that you have the below lines under the scripts.

```
"scripts": {  
  "dev": "nodemon api/index.js",  
  "start": "node api/index.js",  
  "production": "NODE_ENV=production nodemon api/index.js",  
  "test": "echo \"Error: no test specified\" && exit 1",  
  "data:import": "node script/seed.js",  
  "product": "node script/product.js"  
},
```

1. Rename **.env.example** to **.env** (use **.env** not **.env.local** it's only for backend).
2. Configure your MongoDB database, watch this video [MongoDB](#), after configuring you will find a mongo URI just put that on your **.env** file **MONGO_URI** variable.
3. The **JWT_SECRET** is just a random value for creating a secret token, you can use whatever you want but make sure it is secret.
4. You need an **email** and **password** for using email verification and forget the password option. Use an email that you want to send messages to others when they register or request to forget the password. We used Nodemailer and the default email server for this. watch this video to create an [app password](#) for email app-password. After that put your email in the **.env** file **EMAIL_USER** and app password in the **EMAIL_PASS** variable.

Also, you will need to **Allow less secure apps to be ON**, and **access captcha for using this in production environment**, see this [doc](#)

4. Use your local server URL in **STORE_URL** and **ADMIN_URL** variable, when you run on the local server your URL will **http://localhost:3000**, and **http://localhost:4000** .

Finally, your `.env` file will look like this:

```
1
2 PORT=5055
3 MONGO_URI=your mongodb url
4 JWT_SECRET=alamsfdfsdfsdfsdfsdfsdfsdradfar!@#$0fddlfjdfdfdsfsfs
5 JWT_SECRET_FOR_VERIFY=lfjffjasjfsdfsfr09ri09wfsdfsdfri09fdjdj
6
7
8
9
10 SERVICE=gmail
11 EMAIL_USER=your email
12 EMAIL_PASS=your email password
13 HOST=smtp.gmail.com
14 EMAIL_PORT=465
15
16 STRIPE_KEY=your Secret key
17 MAX_AMOUNT=5000.0
18 MIN_AMOUNT =10.0
19 AMOUNT_STEP =5.0
20 CURRENCY=usd
21 STRIPE_PAYMENT_DESCRIPTION="This is Kachabazar payment with Stripe!"
22
23 //when in production mode use this
24 STORE_URL=your store url
25 ADMIN_URL=your admin url
26
```

Once you successfully connect with MongoDB and configure `.env` then run "`npm run data:import`", it will run `seed.js` file and will import all demo data on the database. (You will find all demo data in the `utils` folder, change that data according to your need, also use staff email with real email for use of the forgetting password option) If everything is okay, then the backend configuration is done. Now you will find all demo data in your MongoDB database.

Now run `npm run dev`, it will run your backend on local server on PORT 5055 or your input PORT.

Note: Both Store and Admin backend is the same so use Only one for both

Store Configuration

(If you not purchased kachabazar then omit this configuration)

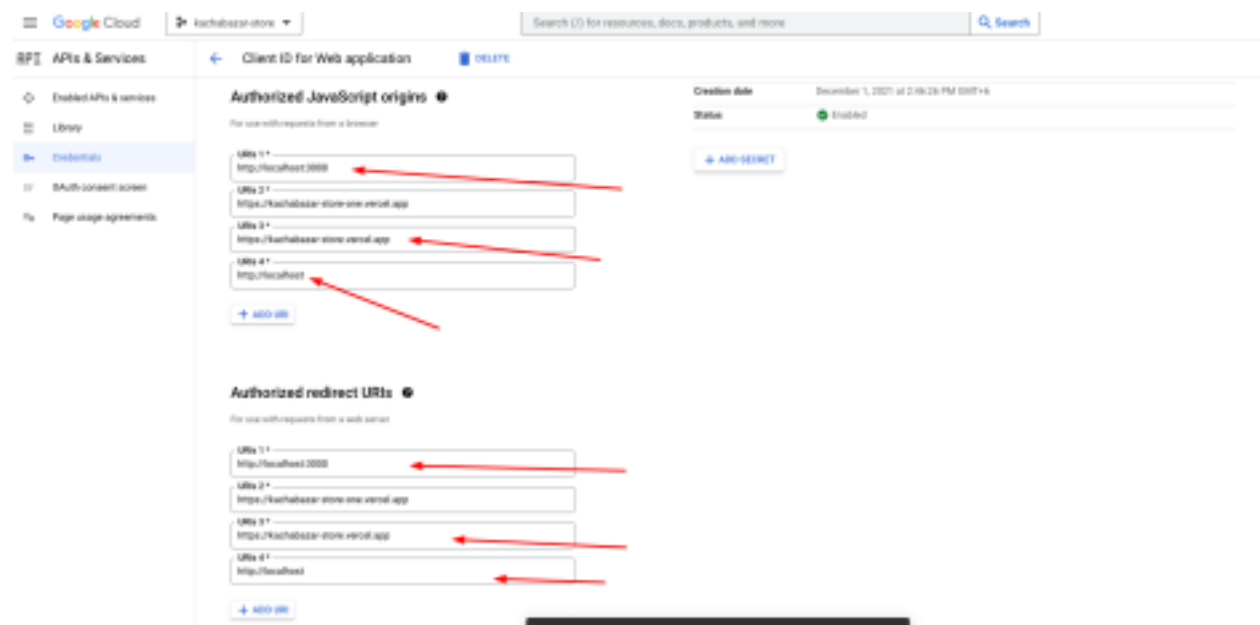
NPM Packages:

- After downloading the template, unzip it
- Open the Terminal /Vs code terminal
- Go to the folder (kachabazar): (`cd path/to/template`)
- Run: `npm install`, it will install all used packages for this kachabazar.
- Make sure that there is no error.

1. Rename `.env.example` to `.env.local`

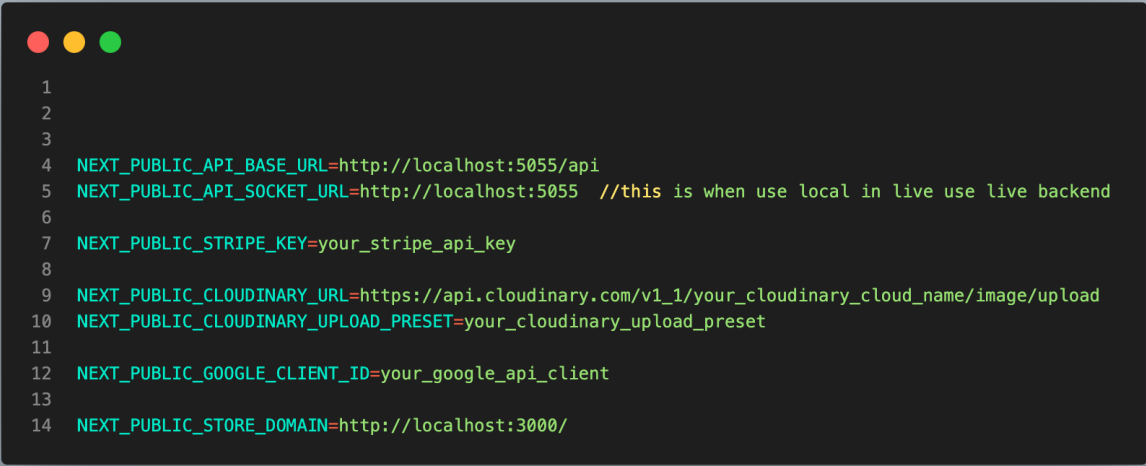
2. Please watch this video for Cloudinary configuration [Cloudinary configuration](#) (We use Cloudinary for profile image upload).

3. This video for google client id [Google Client ID](#) (For google sign in), your google developer console api & services Creadentials will look like this.



4. Also need a stripe API key, for using the stripe payment option. If you want this, then go to [react-stripe](#) and create an account, get your test stripe API key, and put that on the `.env.local` variable. But if you do not add this value on the `.env.local` file then stripe will not work, others will work fine.

After Configure your `.env.local` file will look like this:



```
1
2
3
4 NEXT_PUBLIC_API_BASE_URL=http://localhost:5055/api
5 NEXT_PUBLIC_API_SOCKET_URL=http://localhost:5055 //this is when use local in live use live backend
6
7 NEXT_PUBLIC_STRIPE_KEY=your_stripe_api_key
8
9 NEXT_PUBLIC_CLOUDINARY_URL=https://api.cloudinary.com/v1_1/your_cloudinary_cloud_name/image/upload
10 NEXT_PUBLIC_CLOUDINARY_UPLOAD_PRESET=your_cloudinary_upload_preset
11
12 NEXT_PUBLIC_GOOGLE_CLIENT_ID=your_google_api_client
13
14 NEXT_PUBLIC_STORE_DOMAIN=http://localhost:3000/
```

Now run `npm run dev`, it will run your kachabazar on local server on <http://localhost:3000>

Admin Configuration

(If you not purchased dashtar then omit this configuration)

NPM Packages:

- After downloading the template, unzip it
- Open the Terminal /Vs code terminal
- Go to the folder (dashtar): (`cd path/to/template`)
- Run: `npm install`, it will install all used packages for this dashtar.
- Make sure that there is no error.

1. Rename `.env.example` to `.env.local`

2. Please watch this video for Cloudinary configuration [Cloudinary configuration](#). (We use Cloudinary for image upload). Check bellow screenshot as well.

← Settings

My profile
Account
Users
Product environment settings
Upload
Optimization
Security
Tools
Add-ons

Notification URL:
Send notifications to the specified URL after the completion of actions involving a call to any API methods that support the notification_url parameter. If the notification_url parameter is specified in an API call, that overrides this global setting.

Default Public ID:
Random string (default)
The method to use for assigning a Public ID to an uploaded asset if no Public ID has been specified directly.

Upload presets:

Unsigned uploading enabled

Name	Mode	Settings
[Redacted]	Unsigned	Use filename or externally defined Public ID: false
[Redacted]	Signed	Overwrite: true Use filename or externally defined Public ID: true Unique filename: true

[Add upload preset](#)
Upload presets allow you to define the default behavior for your uploads, instead of receiving these as parameters during the upload request itself. Parameters can include tags, incoming or on-demand transformations, notification URL, and more. Upload presets have precedence over client-side upload parameters.

Product
Programmable Media
Getting Started
Dashboard
Media Library
Media Explorer
Transformations
Reports

Getting Started with Cloudinary

Try your first end-to-end use case

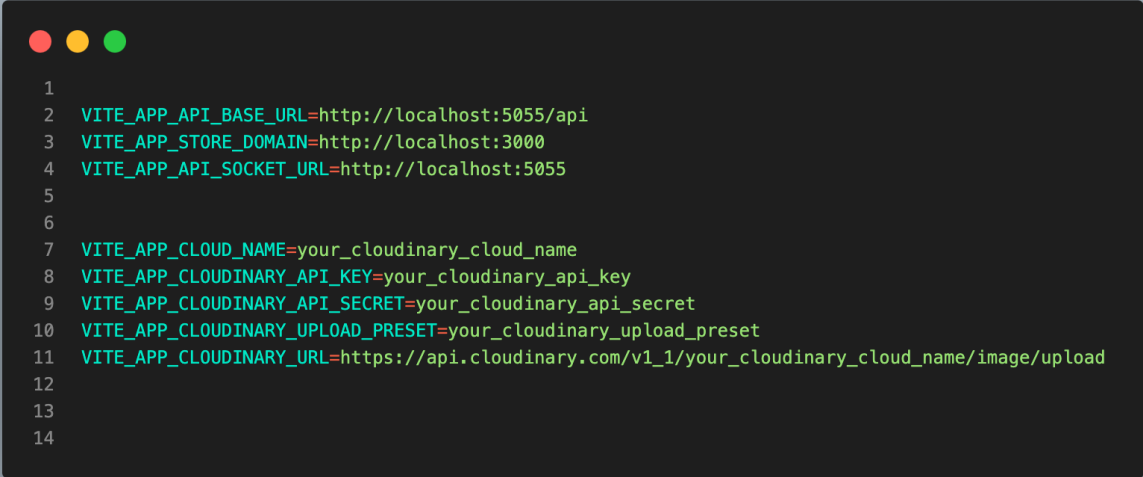
Pick your favorite language and then copy and run the code in the steps to

Node.js React Angular JS Python PHP Java Ruby .NET iOS Android

```
npm install cloudinary
```

```
const cloudinary = require('cloudinary').v2;  
  
// Configuration  
cloudinary.config({  
  cloud_name: '[Redacted]',  
  api_key: '[Redacted]',  
  api_secret: '[Redacted]',  
});
```

After Configure your `.env.local` file will look like this:



```
1
2 VITE_APP_API_BASE_URL=http://localhost:5055/api
3 VITE_APP_STORE_DOMAIN=http://localhost:3000
4 VITE_APP_API_SOCKET_URL=http://localhost:5055
5
6
7 VITE_APP_CLOUD_NAME=your_cloudinary_cloud_name
8 VITE_APP_CLOUDINARY_API_KEY=your_cloudinary_api_key
9 VITE_APP_CLOUDINARY_API_SECRET=your_cloudinary_api_secret
10 VITE_APP_CLOUDINARY_UPLOAD_PRESET=your_cloudinary_upload_preset
11 VITE_APP_CLOUDINARY_URL=https://api.cloudinary.com/v1_1/your_cloudinary_cloud_name/image/upload
12
13
14
```

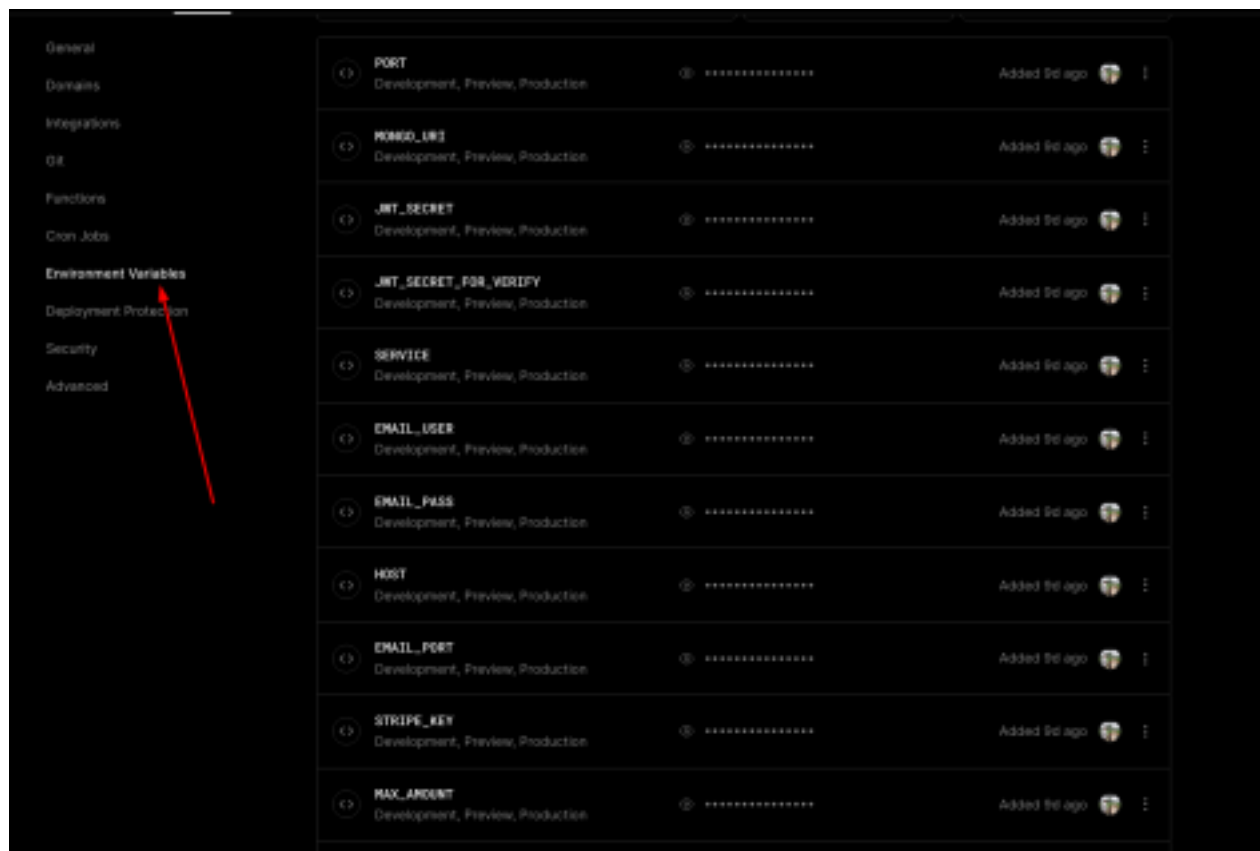
Now run `npm run dev`, it will run your dashtar on local server on <http://localhost:4100> or whatever port you add on package.json dev script.

Deploy On Vercel

As a next.js project vercel is recommended for deployment because if you deploy on vercel then it will automatically do everything for us and there will be no need for customization.

Here is your guide for deploying **KachaBazar** and **Dashtar** on vercel:

1. Create a GitHub account, go to [vercel](https://vercel.com) and sign up with that GitHub account.
2. Create three private repositories on GitHub, then push your **kachabazar** code in one and **backend code** in one, and **admin code** in another repository.
3. Watch this video [deploy on vercel](#), do according to.
4. When you import your GitHub repository on vercel by creating a project, you will see an option for **Environment Variables**, just click on that and give you a local **.env** all variable with the value. then click on deploy. **Note first you will need to import and deploy backend, so that you can use that backend live url as kachabazar and dashtar base url.**



First you have to deploy a backend project.

5. After the backend is deployed successfully, you will find a URL for your API route that will like this <https://kachabazar.vercel.app/>, and now change that like this <https://kachabazar.vercel.app/api> and use this as a `NEXT_PUBLIC_API_BASE_URL` and `VITE_APP_API_BASE_URL` when you deploy your `store` and `admin` project.

6. Now create another two projects for `store` and `admin`. Deploy one by one same as backend project and put all `.env.local` variable before clicking on deploy button, then click deploy, it will take some time for build and after that build, you will see you live version of Kacha Bazar store and Dashtar admin.

7. If you do accordingly, then everything will be okay, for now when you make any changes on your local file, you just need to push your code on GitHub, vercel will automatically detect those changes and will redeploy your project with updated features.

You will find many videos on youtube and also articles on google about how to deploy next.js and express apps on vercel.